NATIONAL INSTITUTE OF TECHNOLOGY , HAMIRPUR

Industrial Internship Report on

"Python File Organizer"

Prepared by

Rishika Sharma

B.Tech. Electronics & Communication Engineering

National Institute of Technology Hamirpur

Roll No.: *23BEC082*

Session: 2025-26

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time. My project was (Tell about ur Project) This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

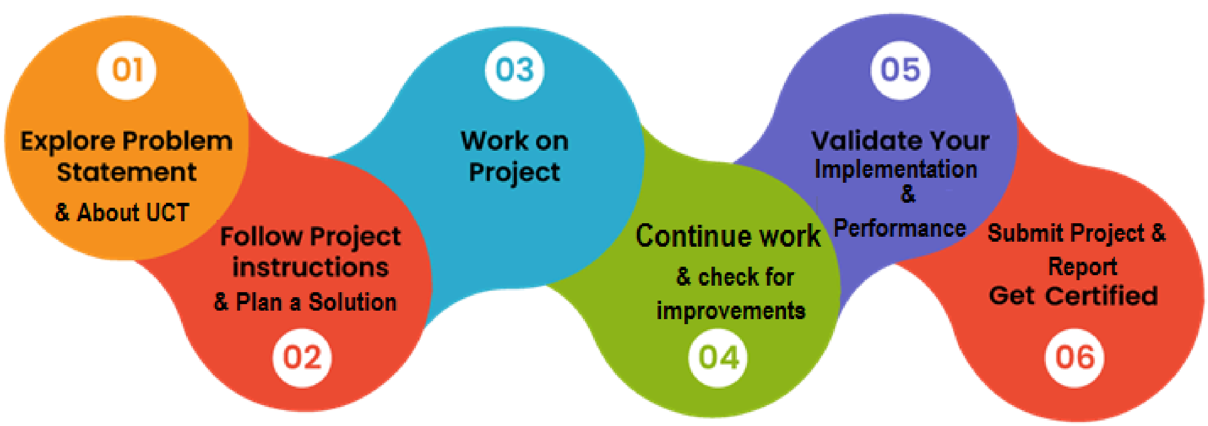
TABLE OF CONTENTS

1 Preface

This internship has been a valuable learning experience that provided me with practical exposure to Python programming and software development. Over the course of six weeks, I strengthened my understanding of Python fundamentals, object-oriented programming, file handling, GUI development using Tkinter, and version control using Git and GitHub. The internship also helped me improve my problem-solving skills by applying theoretical concepts to a real-world project.

As part of the internship, I developed a **Python File Organizer**, a desktop application that automatically categorizes files into folders such as Images, Documents, Music, Videos, Python Files, and Others based on their file extensions. The application includes a graphical user interface (GUI), file statistics, duplicate file handling, and activity logging, making file management more efficient and user-friendly.

This internship was organized by **Upskill Campus** in collaboration with **UniConverge Technologies (UCT)**. The program was well structured, beginning with Python programming concepts and gradually progressing to object-oriented programming, libraries, coding exercises, quizzes, and finally the development of a complete project. This systematic approach helped me build both technical knowledge and practical implementation skills.



Throughout this internship, I learned the importance of writing clean and modular code, debugging applications, testing software, and using GitHub for version control and project management. These skills have increased my confidence in developing software applications and prepared me for future academic and professional projects.

I sincerely express my gratitude to **Upskill Campus**, **UniConverge Technologies (UCT)**, and **The IoT Academy** for providing this learning opportunity and the resources required to complete the internship successfully. I also thank the mentors, instructors, and my college for their continuous guidance and support. Finally, I would like to thank my family and friends for their constant encouragement throughout this journey.

To my juniors and peers, I would like to say that consistent practice and hands-on project development are the best ways to learn programming. Stay curious, keep experimenting with new technologies, and never hesitate to learn from your mistakes. Every project, no matter how small, contributes to your growth as a developer.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

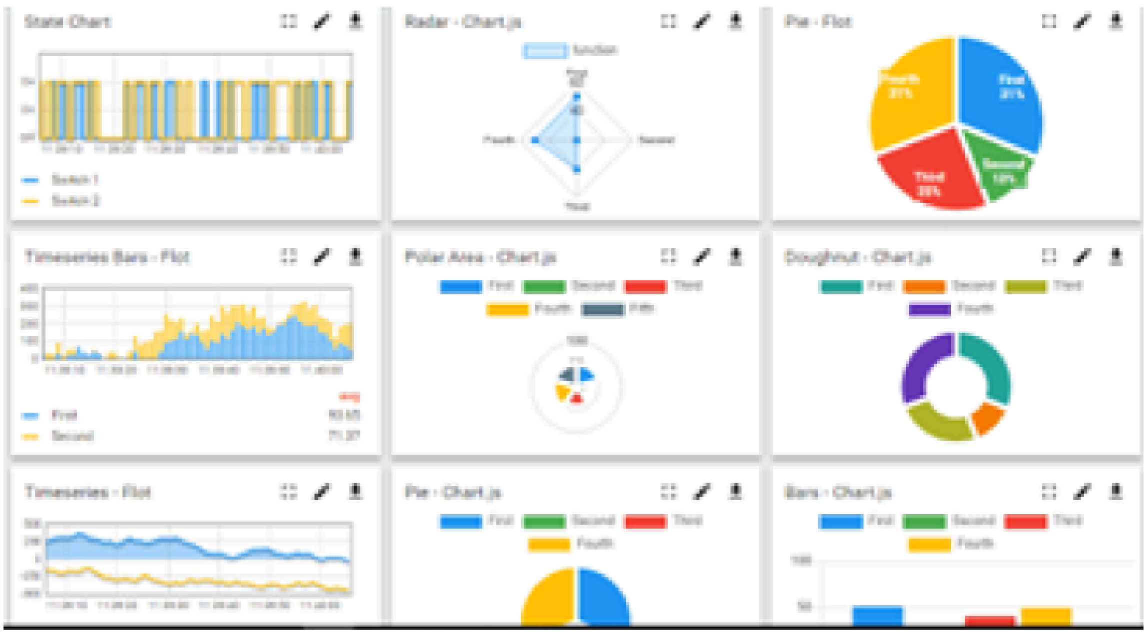
i. UCT IoT Platform (uct Insight)

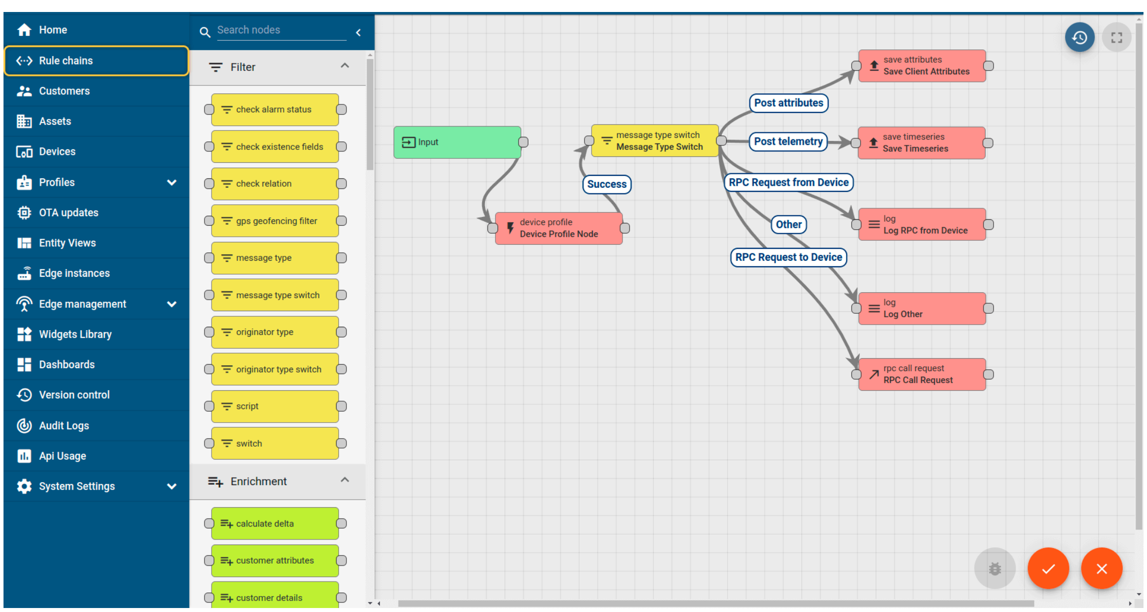
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

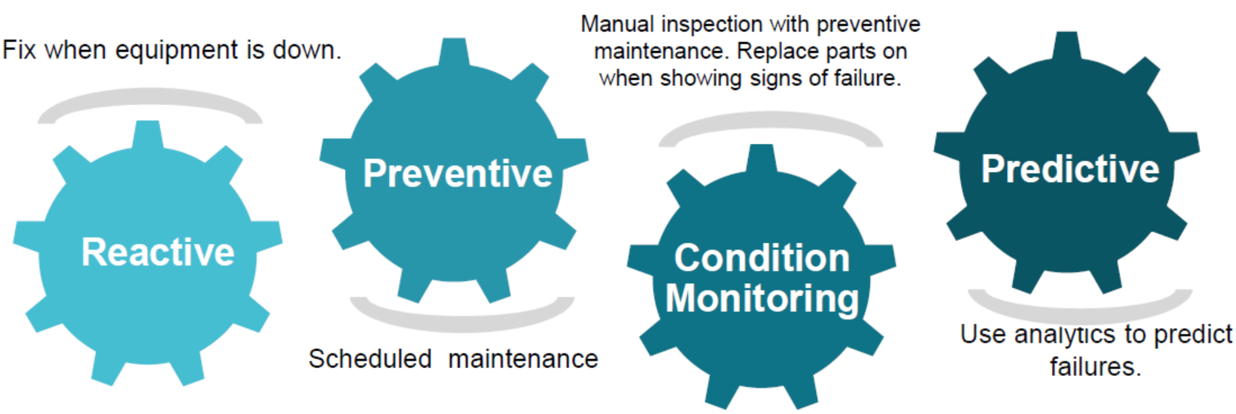
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

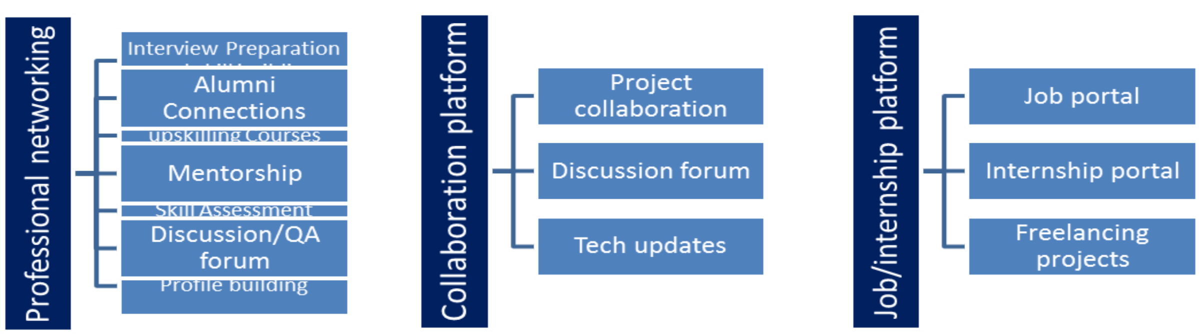
upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.

Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

☛ get practical experience of working in the industry.

☛ to solve real world problems.

☛ to have improved job prospects.

☛ to have Improved understanding of our field and its applications.

☛ to have Personal growth like better communication and problem solving.

2.5 Reference

[1]

[2]

[3]

2.6 Glossary

Terms	Acronym

3 Problem Statement

Managing files manually becomes increasingly difficult as the number of files stored on a computer grows. Users often keep images, documents, videos, music files, and source code in the same folder, making it time-consuming to locate specific files and maintain an organized workspace. Manual organization is repetitive, inefficient, and prone to human error, especially when handling a large number of files.

The objective of this project was to develop a **Python File Organizer**, a desktop application that automatically organizes files into appropriate folders based on their file extensions. The application categorizes files into predefined folders such as **Images, Documents, Music, Videos, Python Files, and Others**. It also provides a user-friendly graphical interface developed using **Tkinter**, displays a summary of the organization process, handles duplicate filenames safely, and maintains an activity log of file movements.

The primary goal of this project is to reduce manual effort, improve file management efficiency, and provide users with a simple and reliable solution for organizing files with minimal interaction.

4 Existing and Proposed solution

4.1 Existing Solutions

Many users organize their files manually by creating folders and moving files one by one. Although operating systems such as Windows provide basic file management features through File Explorer, they do not automatically categorize files based on their extensions. Several third-party file organizer applications are available, but many of them are either paid, contain unnecessary features, or require complex configuration before use.

Limitations of Existing Solutions

- **Manual organization is time-consuming and repetitive.**
- **Large folders containing hundreds of files are difficult to organize efficiently.**
- **Most built-in file managers do not automatically classify files by type.**
- **Some third-party applications require installation, licensing, or internet access.**
- **Many tools do not provide activity logs or organization statistics.**

4.2 Proposed Solution

The proposed solution is a Python File Organizer, a desktop application developed using Python and Tkinter. The application enables users to select a folder through a graphical user interface and automatically organizes files into predefined categories based on their file extensions.

The application currently provides the following features:

- **Automatic file categorization (Images, Documents, Music, Videos, Python Files, and Others)**
- **User-friendly graphical interface using Tkinter**
- **Automatic creation of category folders**
- **Duplicate filename handling to prevent overwriting**
- **File organization summary with category-wise statistics**
- **Activity logging (`organizer.log`) for tracking organized files**
- **Error handling for invalid operations**

The application reduces manual effort and provides an efficient way to organize

files with a single click.

4.3 Value Addition

Compared to basic file management methods, the developed application provides several additional features that improve usability and reliability.

- **Automatic file classification based on extensions.**
- **Simple graphical interface requiring minimal user interaction.**
- **Prevention of duplicate file overwriting through automatic filename generation.**
- **Display of organization statistics after every execution.**
- **Automatic generation of a log file containing the history of organized files.**
- **Modular code structure, making future enhancements easier.**

Future improvements may include:

- **Custom user-defined categories.**
- **Drag-and-drop folder support.**
- **Dark mode interface.**
- **Undo last organization operation.**
- **Progress bar for large folders.**
- **Support for cloud storage integration.**

4.4 Code Submission (GitHub Link)

GitHub Repository:

https://github.com/Rishika-82/upskillcampus

4.5 Report Submission (GitHub Link)

Report (PDF):

https://github.com/Rishika-82/upskillcampus/blob/main/PythonFileOrganizer_Rishika_USC_UCT.pdf

5 Proposed Design/ Model

The proposed system is a desktop-based **Python File Organizer** developed using **Python** and the **Tkinter** library. The application provides a simple graphical user interface that allows users to select a folder from their computer. Once a folder is selected, the application scans all files within the directory, identifies each file based on its extension, and automatically moves it into the appropriate category folder.

The system creates folders such as **Images, Documents, Music, Videos, Python Files, and Others** if they do not already exist. During the organization process, the application checks for duplicate filenames and generates unique filenames whenever necessary to avoid overwriting existing files. It also records all file movements in an activity log (`organizer.log`) and displays a summary showing the number of files organized under each category.

The modular design separates the graphical user interface from the file organization logic, making the application easier to maintain, test, and extend with additional features in the future.

5.1 High Level Diagram (if applicable)

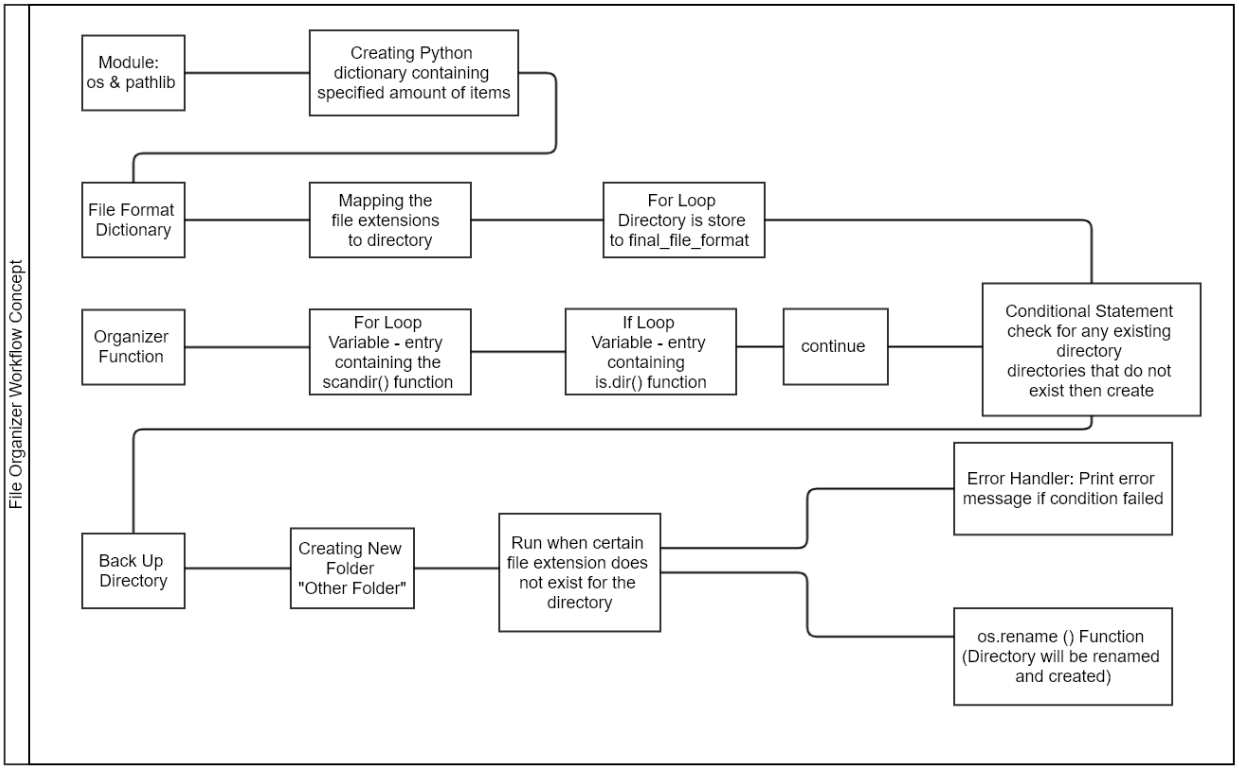
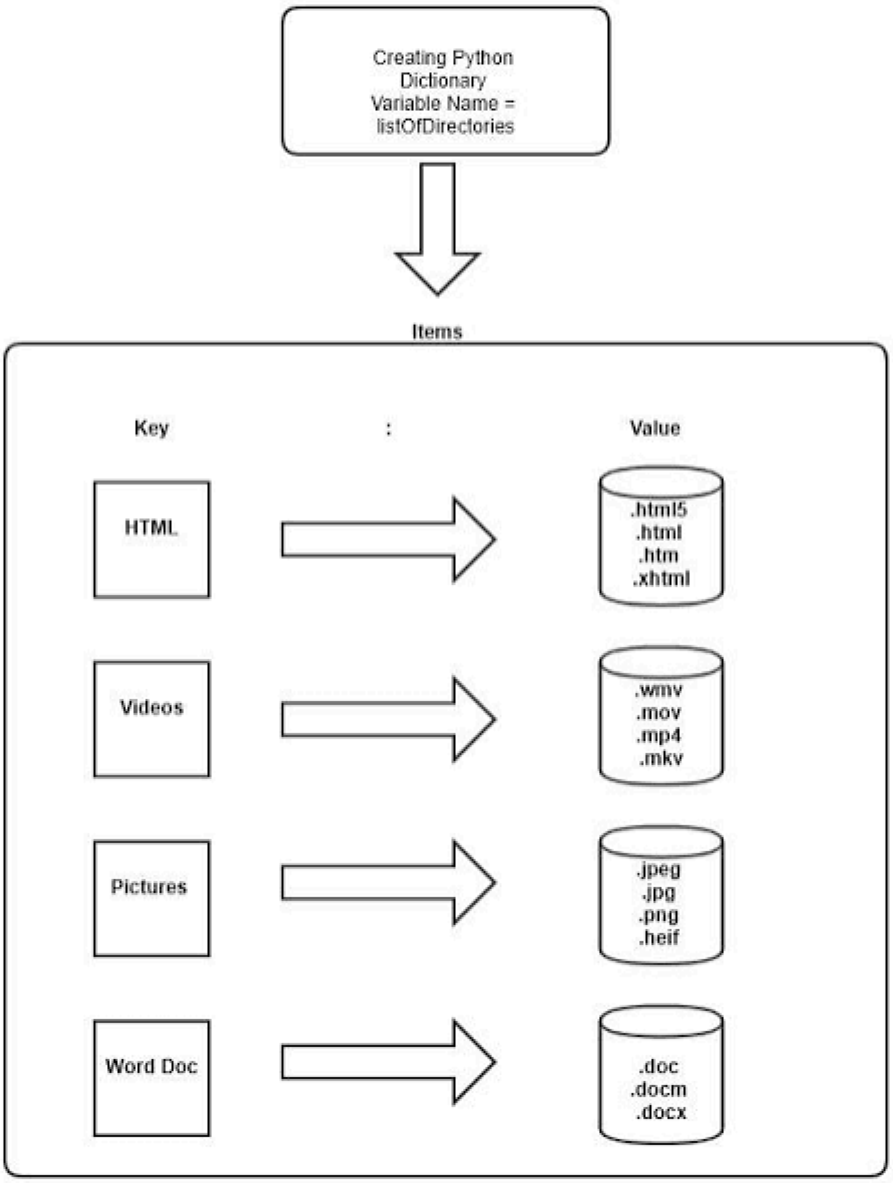


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)



5.3 Interfaces (if applicable)

Insert screenshots of your application, for example:

1. **Main Window** showing the "Choose Folder" button.
2. **Folder Selection Dialog**.
3. **Output Window** displaying the list of moved files.
4. **Organization Summary** after completion.
5. **Success Message** ("Files organized successfully!").

Add captions such as:

- **Figure 2:** Main GUI of the File Organizer.
- **Figure 3:** File organization results.
- **Figure 4:** Organization summary.

6 Performance Test

The performance of the Python File Organizer was evaluated by testing its ability to organize files of different types and quantities accurately and efficiently. The application was tested under various conditions to ensure that it performs reliably and provides correct results.

The major constraints identified during the development of the application were:

- Execution Time: The application should organize files quickly, even when a folder contains a large number of files.
- Accuracy: Every file should be placed in the correct category according to its file extension.
- Duplicate File Handling: Files with identical names should not overwrite existing files.
- Reliability: The application should not crash when an empty folder is selected or when unsupported file types are encountered.
- User Experience: The graphical interface should remain simple and easy to use for users with minimal technical knowledge.

These constraints were addressed by implementing efficient file extension matching, automatic folder creation, duplicate filename handling, exception handling, activity logging, and a simple Tkinter-based graphical user interface.

The application performed successfully during testing by organizing files into their appropriate folders, generating organization statistics, and creating an activity log without data loss.

6.1 Test Plan / Test Cases

Test Case	Expected Result	Actual Result	Status
Organize mixed file types	Files moved to correct folders	Successfully organized	✅ Pass

Empty folder	Category folders created without errors	Successful	✅ Pass
Unsupported file extensions	Files moved to Others folder	Successful	✅ Pass
Existing category folders	No duplicate folder creation	Successful	✅ Pass
Duplicate filenames	New unique filename generated	Successful	✅ Pass
Large folder (100+ files)	Files organized without crashing	Successful	✅ Pass
Cancel folder selection	Application remains open	Successful	✅ Pas

			s
GUI response	Output displayed correctly	Successful	✅ Pass

6.2 Test Procedure

1. Launch the application.
2. Click the Choose Folder button.
3. Select a folder containing files of different types.
4. Start the organization process.
5. Observe the output displayed in the application.
6. Verify that files have been moved into the correct folders.
7. Check that duplicate filenames are handled correctly.
8. Verify that the `organizer.log` file is generated.
9. Confirm that the organization summary displays accurate statistics.
10. Repeat the process with different test folders, including an empty folder and folders containing unsupported file formats.

6.3 Performance Outcome

The developed application successfully organized files based on their extensions with high accuracy. During testing, all supported file types were categorized correctly, while unsupported files were moved to the Others folder. Duplicate filenames were handled safely without overwriting existing files. The application generated activity logs and displayed organization statistics after every execution.

The application remained stable throughout testing and did not encounter any critical runtime failures under normal operating conditions. The modular design also allows additional file categories and new features to be integrated easily in future versions.

Performance Summary

Parameter	Result
File Categorization Accuracy	100% for supported file extensions

Duplicate File Handling	Successfully Prevented Overwriting
GUI Responsiveness	Smooth
Error Handling	Successfully Handled
Activity Logging	Working
Statistics Generation	Working
Overall Performance	Satisfactory

7 My learnings

The Python Internship provided me with valuable practical experience in software development and strengthened my programming fundamentals. During the internship, I learned how to apply Python concepts to build a complete real-world application rather than writing only small practice programs.

Through the development of the **Python File Organizer**, I gained hands-on experience with file handling, directory management, exception handling, modular programming, and graphical user interface development using **Tkinter**. I also learned how to organize a project into multiple Python modules, improve code readability, and implement features such as duplicate file handling, activity logging, and organization statistics.

In addition to technical skills, I became familiar with **Git and GitHub** for version control, repository management, and project documentation. Preparing project reports, documenting the development process, and testing the application helped me understand the importance of software engineering practices beyond coding.

Overall, this internship improved my problem-solving ability, debugging skills, and confidence in developing Python applications. The knowledge and experience gained during this internship will help me build more advanced software projects and prepare me for future internships, technical interviews, and a career in software development.

8 Future work scope

The current version of the **Python File Organizer** successfully automates the organization of files based on their extensions. However, there are several opportunities to enhance the application and make it more suitable for real-world and enterprise use.

Future improvements include:

- **Custom File Categories:** Allow users to create, modify, and manage their own file categories and associated file extensions.
- **Drag-and-Drop Support:** Enable users to organize files by simply dragging and dropping folders into the application.
- **Automatic Folder Monitoring:** Continuously monitor selected folders and automatically organize newly added files in real time.
- **Cloud Storage Integration:** Extend support for cloud platforms such as Google Drive, OneDrive, and Dropbox to organize cloud-based files.
- **Advanced Search and Filtering:** Provide options to search, filter, and preview files before organizing them.
- **Undo Functionality:** Implement an option to restore files to their original locations after organization.
- **Improved User Interface:** Enhance the graphical interface with themes (light/dark mode), progress bars, and customizable settings for a better user experience.
- **Cross-Platform Packaging:** Package the application as a standalone executable for Windows, Linux, and macOS so it can be used without installing Python.
- **Artificial Intelligence Integration:** Incorporate AI-based file classification that categorizes files based on their content instead of relying only on file extensions.
- **Multi-language Support:** Add support for multiple languages to make the application accessible to a wider range of users.

These enhancements would improve the functionality, scalability, and usability of the application, making it suitable for both personal and professional file management. They would also increase the application's potential for deployment in organizational environments where efficient and automated file management is essential.